

UNIDAD 7 · SEMANAS 10–12

Hilos y concurrencia

📅 Semanas 10–12 · fechas por confirmar </> Laboratorio TD6 (resuelto + desafío IA)

~ 5 animaciones

MANUAL M7 · TEORÍA DE LA UNIDAD

Hilos y concurrencia

Lectura base de la unidad 7 (≈ 2 h). Las salidas son reales: programas del TD6 ejecutados en el equipo del curso (28 núcleos x86-64).

CONTENIDO

§1	Hilos	§4	Mutex, semáforos y variables de condición
§2	Condiciones de carrera y atomicidad		
§3	Exclusión mutua: del software al hardware	§5	Interbloqueos

§1 Hilos

Un **hilo** es una ejecución secuencial dentro de un proceso. Los hilos de un proceso comparten todo el espacio de direcciones —código, datos globales, heap— y los descriptores; cada uno tiene su propia **pila**, sus **registros** y su información de planificación. No hay protección de memoria entre ellos: un error de un hilo puede corromper los datos de todos, y una señal como SIGSEGV termina el proceso entero. En Linux, el núcleo planifica cada hilo como una tarea más (se crean con `clone`, compartiendo la memoria y los descriptores).

La API POSIX: `pthread_create(&t, attr, f, arg)` lanza `f(arg)` en un hilo nuevo; `pthread_exit` (o volver de `f`) lo termina; `pthread_join` espera su fin, recoge el valor de retorno y libera sus recursos (o `pthread_detach`, si nadie lo esperará). Estas funciones devuelven 0 o un número de error, no -1. Se compila con `-pthread`. Crear un hilo es mucho más barato que crear un proceso porque no se copian tablas de páginas:

Salida real

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

Salida real

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5, local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2, local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3, local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6, local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7, local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8, local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8
```

Practica esto → A7.4 hilos frente a procesos · TD6 · 1

§2 Condiciones de carrera y atomicidad

Una línea de C como `x = x + 2` son varios pasos del procesador: leer `x` a un registro, sumar y escribir. Si dos hilos ejecutan líneas así sobre la misma variable, el planificador puede intercalar sus pasos de cualquier manera, y en algunos órdenes una escritura **pisa** a la otra: es una **condición de carrera**. Para `x = x+2 || x = x*3` con `x = 1` hay 20 intercalados posibles; solo los dos en serie dan 9 o 5, el resto pierde una actualización. En la práctica los órdenes "malos" son poco frecuentes, lo que vuelve a estos errores difíciles de reproducir: en el TD6, ocho hilos vendiendo 100 000 boletos sin protección venden algunos miles de más.

Una operación es **atómica** si se ejecuta entera o no se ejecuta, sin que otro hilo vea un estado intermedio. Desde C11, dos accesos a la misma variable desde hilos distintos, sin sincronización y con al menos una escritura, son una *carrera de datos* y el comportamiento es indefinido; `volatile` no lo arregla. Para operaciones atómicas se usan `<stdatomic.h>` o `__atomic_*`.

Salida real

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

Practica esto → A7.1 condición de carrera · TD6 · 2

§3 Exclusión mutua: del software al hardware

Una **sección crítica** es un fragmento que solo un hilo puede ejecutar a la vez (**exclusión mutua**). El problema de "demasiada leche" muestra lo difícil que es lograrlo solo con lecturas y escrituras: mirar y comprar deja una ventana; una nota compartida también (mirarla y ponerla son dos

pasos); una nota por hilo evita comprar dos veces pero permite que nadie compre. El **algoritmo de Peterson** (nota propia + ceder el turno + esperar mientras el otro tenga nota y sea su turno) es correcto si la memoria es *secuencialmente consistente*.

Los procesadores reales no lo son. En x86, cada núcleo deja sus escrituras en un **búfer** y puede leer la memoria antes de que ellas sean visibles para los demás (modelo TSO); ARM reordena todavía más. Así, los dos hilos de Peterson pueden leer la nota del otro como 0 y entrar a la vez. Medido en el equipo del curso, con y sin barrera de memoria (`mfence`):

Salida real

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

Por eso la exclusión mutua se construye con **instrucciones atómicas del hardware**: *test-and-set*, *compare-and-swap* (`lock cmpxchg` en x86), *fetch-and-add*. Un cerrojo que espera en un bucle (*spinlock*) gasta CPU y puede sufrir *inversión de prioridad*; la solución de Linux es el **futex**: si el cerrojo está libre se toma con una instrucción atómica en espacio de usuario, y solo si hay que esperar se llama al núcleo para dormir. Sin competencia, un mutex prácticamente no hace llamadas al sistema:

Salida real

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex
./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00 0.000000 0 1 futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00 0.223730 27 8013 3116 futex
```

Practica esto → A7.5 el problema de la leche · TD6 · 5

§4 Mutex, semáforos y variables de condición

Un **mutex** (`pthread_mutex_lock` / `unlock`) protege una sección crítica: quien encuentra el mutex tomado duerme hasta que se libere. Un **semáforo** (Dijkstra) es un entero ≥ 0 con dos

operaciones atómicas: `sem_wait` (P: espera a que sea positivo y resta 1) y `sem_post` (V: suma 1 y despierta a un hilo en espera). Iniciado en 1 es un cerrojo; en 0, un aviso entre hilos; en k, un límite de k usuarios a la vez. `sem_init` crea uno anónimo (compartible entre procesos con `pshared = 1` en memoria compartida) y `sem_open` uno con nombre (devuelve `SEM_FAILED` si falla).

El **productor-consumidor** combina ambos: un búfer de `BUF` celdas, un semáforo `empty` (celdas libres, inicia en `BUF`), otro `full` (celdas ocupadas, inicia en 0) y un mutex para los índices compartidos. El productor espera si el búfer está lleno y el consumidor si está vacío; nada se pierde ni se repite:

Salida real

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

Para esperar una condición arbitraria se usan **variables de condición**: con el mutex tomado, `while (!condición) pthread_cond_wait(&c, &m);`. `cond_wait` suelta el mutex y duerme de forma atómica, y lo vuelve a tomar al despertar; quien cambia la condición llama a `pthread_cond_signal` o `_broadcast`. Se usa `while` porque puede haber despertares espurios.

Practica esto → A7.2 productor-consumidor · TD6 · 3

§5 Interbloqueos

Un **interbloqueo** es un conjunto de hilos que se esperan mutuamente para siempre. Aparece solo si se cumplen **las cuatro condiciones** (Coffman): exclusión mutua, retener recursos mientras se esperan otros, ausencia de expropiación y espera circular. Como depende del orden de ejecución, puede no aparecer en miles de pruebas. Los filósofos comensales lo muestran: si cada uno toma primero el palillo derecho y luego el izquierdo, todos pueden quedar con uno esperando el del vecino. Basta romper una condición: un **orden global** de adquisición (la solución del TD6), limitar cuántos compiten (un "camarero"), soltar y reintentar (`trylock`), o detectar ciclos en el grafo de asignación y recuperarse. En el núcleo de Linux, *lockdep* vigila que los cerrojos se tomen siempre en el mismo orden.

Salida real

```

$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status: ${PIPESTATUS[0]}"
(124 = still blocked after 3 s)
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)

```

Salida real

```

$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals

```

Practica esto → A7.3 los filósofos · TD6 · 4